03 - Etapas: Implementación de MCP Unity

Resumen de Etapas

```
Etapa 1: Setup y Infra
  ├─ Crear proyecto Node.js/TypeScript
  ├─ Instalar dependencias MCP
  └─ Estructura base de servidor

Etapa 2: Herramientas Básicas (Fase I)
  ├─ CreateScript
  ├─ DeleteScript
  └─ GetErrors

Etapa 3: Herramientas de GameObjects (Fase II)
  ├─ CreateGameObject
  ├─ ConfigureComponent
  └─ DeleteGameObject

Etapa 4: Herramientas de Prefabs (Fase III)
  ├─ CreatePrefab
  ├─ ReadPrefab
  ├─ WritePrefab
  └─ DeletePrefab

Etapa 5: Compilación y Build (Fase IV)
  ├─ CompileProject
  ├─ GetCompileErrors
  ├─ BuildGame (Standalone)
  └─ GetBuildStatus
```

ETAPA 1: Setup e Infraestructura

1.1 Crear estructura base de proyecto

```
# En MCP/servers/unity/
mkdir -p src/{tools,utils} build

# Crear archivos base
touch package.json
touch tsconfig.json
touch src/index.ts
touch src/unity-cli.ts
touch src/types.ts
```

1.2 package.json

```
{
  "name": "zoremgame-mcp-unity",
  "version": "1.0.0",
  "description": "MCP Server for Unity game development automation",
  "main": "build/index.js",
  "scripts": {
    "build": "tsc",
    "dev": "ts-node src/index.ts",
    "start": "node build/index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "latest",
    "jsonrpc-lite": "^2.3.0",
    "child_process": "latest",
    "fs-extra": "^11.0.0"
  },
  "devDependencies": {
    "@types/node": "^20.0.0",
    "typescript": "^5.0.0",
    "ts-node": "^10.0.0"
  }
}
```

1.3 tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./build",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

1.4 src/types.ts

```
// Tipos base para el MCP Unity Server

export interface ToolInput {
  [key: string]: string | number | boolean | undefined;
}

export interface ToolResult {
  content: Array<{
    type: "text" | "image" | "resource";
    text?: string;
    uri?: string;
    mimeType?: string;
  }>;
}

export interface CompileResult {
  success: boolean;
  errors: string[];
  warnings: string[];
  duration: number; // ms
}

export interface UnityProjectConfig {
  projectPath: string;
  unityExecutable: string;
  assetPath: string;
  autoCompile: boolean;
}

export interface ScriptCreateInput {
  path: string;
  content: string;
  namespace?: string;
  auto_compile?: boolean;
}

export interface ScriptDeleteInput {
  path: string;
}

export interface CompileInput {
  target?: "StandaloneWindows64" | "StandaloneLinux64" | "StandaloneOSX";
}
```

1.5 src/unity-cli.ts (CLI Wrapper)

```
// Interfaz con Unity Editor via CLI

import { exec } from "child_process";
import { promisify } from "util";
```

```
import * as fs from "fs-extra";
import * as path from "path";

const execAsync = promisify(exec);

export class UnityCLI {
  private projectPath: string;
  private unityExecutable: string;

  constructor(projectPath: string, unityExecutable: string) {
    this.projectPath = projectPath;
    this.unityExecutable = unityExecutable;
  }

  /**
   * Ejecuta Unity en batch mode con método específico
   */
  async executeMethod(
    className: string,
    methodName: string,
    timeout: number = 60000
  ): Promise<{ stdout: string; stderr: string }> {
    const command = `"${this.unityExecutable}" \
      -projectPath "${this.projectPath}" \
      -batchmode \
      -quit \
      -executeMethod ${className}.${methodName} \
      -logFile -`;

    try {
      const result = await execAsync(command, {
        timeout,
        maxBuffer: 10 * 1024 * 1024, // 10MB
      });
      return result;
    } catch (error) {
      throw new Error(
        `Unity execution failed: ${error instanceof Error ? error.message :
String(error)}`
      );
    }
  }

  /**
   * Compila el proyecto
   */
  async compile(): Promise<{
    success: boolean;
    errors: string[];
    warnings: string[];
  }> {
    try {
      await this.executeMethod("UnityEditor.EditorBuildSettings", "Refresh");
      return { success: true, errors: [], warnings: [] };
```

```
    } catch (error) {
      return {
        success: false,
        errors: [error instanceof Error ? error.message : String(error)],
        warnings: [],
      };
    }
  }

  /**
   * Obtiene errores de compilación del último build
   */
  async getCompileErrors(): Promise<string[]> {
    const logPath = path.join(this.projectPath, "Logs");
    try {
      const logs = await fs.readdir(logPath);
      // Leer archivos de log más recientes
      const errors: string[] = [];
      // Parsing logic aquí
      return errors;
    } catch {
      return [];
    }
  }

  /**
   * Valida que un script exista y sea válido
   */
  async validateScript(scriptPath: string): Promise<boolean> {
    const fullPath = path.join(this.projectPath, scriptPath);
    return fs.pathExists(fullPath);
  }
}
```

1.6 src/index.ts (Entry Point - Parte 1)

```
// Entry point del MCP Unity Server

import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import {
  CallToolRequestSchema,
  TextContent,
} from "@modelcontextprotocol/sdk/types.js";
import * as fs from "fs-extra";
import * as path from "path";
import { UnityCLI } from "./unity-cli";
import {
  CompileResult,
  ScriptCreateInput,
  ToolResult,
  UnityProjectConfig,
```

```
} from "./types";

// Leer configuración del environment
const config: UnityProjectConfig = {
  projectPath: process.env.UNITY_PROJECT_PATH || "/path/to/ZoremGame",
  unityExecutable:
    process.env.UNITY_EXECUTABLE ||
"/Applications/Unity/Unity.app/Contents/MacOS/Unity",
  assetPath: process.env.ASSET_STORE_PATH || "Assets",
  autoCompile: process.env.AUTO_COMPILE === "true",
};

const cli = new UnityCLI(config.projectPath, config.unityExecutable);
const server = new Server({
  name: "zoremgame-unity-mcp",
  version: "1.0.0",
});

// ============================================
// HERRAMIENTAS (Tools)
// ============================================

/**
 * Tool: CreateScript
 * Crea un nuevo script C# en el proyecto
 */
async function handleCreateScript(
  input: ScriptCreateInput
): Promise<ToolResult> {
  try {
    // Validar entrada
    if (!input.path || !input.content) {
      throw new Error("path and content are required");
    }

    // Sanitizar path
    const safePath = input.path.replace(/\.\.\//g, "");
    const fullPath = path.join(config.projectPath, safePath);

    // Validar que está dentro de Assets
    if (!fullPath.startsWith(path.join(config.projectPath, config.assetPath))) {
      throw new Error("Scripts must be in Assets folder");
    }

    // Validar que no existe
    const exists = await fs.pathExists(fullPath);
    if (exists) {
      throw new Error(`File already exists: ${safePath}`);
    }

    // Crear directorio si no existe
    await fs.ensureDir(path.dirname(fullPath));

    // Obtener namespace default
```

```
    const namespace = input.namespace || "ZoremGame";

    // Envolver código en namespace
    const wrappedContent = wrapInNamespace(input.content, namespace);

    // Escribir archivo
    await fs.writeFile(fullPath, wrappedContent);

    // Compilar si está habilitado
    if (input.auto_compile && config.autoCompile) {
      await cli.compile();
    }

    return {
      content: [
        {
          type: "text",
          text: `Script created successfully at ${safePath}\nNamespace:
${namespace}`,
        },
      ],
    };
  } catch (error) {
    throw new Error(
      `CreateScript failed: ${error instanceof Error ? error.message :
String(error)}`
    );
  }
}

/**
 * Tool: DeleteScript
 * Elimina un script C# del proyecto
 */
async function handleDeleteScript(input: {
  path: string;
}): Promise<ToolResult> {
  try {
    if (!input.path) {
      throw new Error("path is required");
    }

    const safePath = input.path.replace(/\.\.\//g, "");
    const fullPath = path.join(config.projectPath, safePath);

    const exists = await fs.pathExists(fullPath);
    if (!exists) {
      throw new Error(`File not found: ${safePath}`);
    }

    await fs.remove(fullPath);
    await fs.remove(fullPath + ".meta");

    return {
```

```
      content: [
        {
          type: "text",
          text: `Script deleted: ${safePath}`,
        },
      ],
    };
  } catch (error) {
    throw new Error(
      `DeleteScript failed: ${error instanceof Error ? error.message :
String(error)}`
    );
  }
}

/**
 * Tool: GetErrors
 * Obtiene errores de compilación del proyecto
 */
async function handleGetErrors(): Promise<ToolResult> {
  try {
    const errors = await cli.getCompileErrors();
    return {
      content: [
        {
          type: "text",
          text:
            errors.length > 0
              ? `Compilation errors:\n${errors.join("\n")}`
              : "No compilation errors",
        },
      ],
    };
  } catch (error) {
    throw new Error(
      `GetErrors failed: ${error instanceof Error ? error.message :
String(error)}`
    );
  }
}

// ==========================================
// UTILIDADES
// ==========================================

function wrapInNamespace(content: string, namespace: string): string {
  const trimmed = content.trim();

  // Si ya tiene namespace, no envolver
  if (trimmed.includes("namespace ")) {
    return content;
  }

  return `namespace ${namespace}\n{\n${content}\n}\n`;
```

```
}

// ============================================
// SETUP DEL SERVER
// ============================================

server.setRequestHandler(CallToolRequestSchema, async (request) => {
  switch (request.params.name) {
    case "create_script":
      return await handleCreateScript(
        request.params.arguments as ScriptCreateInput
      );

    case "delete_script":
      return await handleDeleteScript(
        request.params.arguments as { path: string }
      );

    case "get_errors":
      return await handleGetErrors();

    default:
      throw new Error(`Unknown tool: ${request.params.name}`);
  }
});

// ============================================
// INICIAR SERVIDOR
// ============================================

async function main() {
  console.error(
    "[Unity MCP] Server starting with config:",
    JSON.stringify(config, null, 2)
  );

  // Validar que proyecto existe
  const projectExists = await fs.pathExists(
    path.join(config.projectPath, "ProjectSettings")
  );
  if (!projectExists) {
    console.error("[Unity MCP] ERROR: Unity project not found at",
config.projectPath);
    process.exit(1);
  }

  console.error("[Unity MCP] Project found, server ready");
  await server.connect(process.stdin, process.stdout);
}

main().catch((error) => {
  console.error("Fatal error:", error);
  process.exit(1);
});
```

ETAPA 2: Herramientas Básicas (Fase I)

Esta etapa está **parcialmente cubierta** en 1.6 (CreateScript, DeleteScript, GetErrors).

2.1 Registrar herramientas en el servidor

En `src/index.ts`, agregar el handler de herramientas (ya incluido arriba).

2.2 Testing de herramientas básicas

```
# Compilar
npm run build

# Ejecutar servidor en modo debug
AUTO_COMPILE=false npm start

# En otra terminal, prueba:
# (Requiere estar en formato MCP protocol)
```

ETAPA 3: Herramientas de GameObjects (Fase II)

Agregar a `src/tools/gameobjects.ts`:

```
// Truncado por brevedad - incluir en archivo completo
// HandleCreateGameObject
// HandleConfigureComponent
// HandleDeleteGameObject
```

ETAPA 4: Herramientas de Prefabs (Fase III)

Agregar a `src/tools/prefabs.ts`:

```
// Truncado por brevedad
// HandleCreatePrefab
// HandleReadPrefab
// HandleWritePrefab
```

ETAPA 5: Compilación y Build (Fase IV)

Agregar a `src/tools/build.ts`:

```
// HandleCompile
// HandleBuild
// HandleGetErrors
```

Checklist de Implementación

Etapa 1 ☑

- ☐ Crear estructura de carpetas
- ☐ Setup package.json y tsconfig.json
- ☐ Crear types.ts
- ☐ Crear unity-cli.ts
- ☐ Crear index.ts básico
- ☐ Compilar sin errores
- ☐ Verificar que ejecuta sin crash

Etapa 2 ☑

- ☐ CreateScript funciona
- ☐ DeleteScript funciona
- ☐ GetErrors funciona
- ☐ Testing manual de cada tool

Etapa 3

- ☐ CreateGameObject funciona
- ☐ ConfigureComponent funciona
- ☐ Validación de componentes

Etapa 4

- ☐ CreatePrefab funciona
- ☐ ReadPrefab funciona
- ☐ WritePrefab funciona

Etapa 5

- ☐ CompileProject funciona
- ☐ BuildGame funciona
- ☐ Error reporting funciona
- ☐ Integración con claude code

Próximas Etapas

Después de completar todas las etapas de Unity MCP, sigue a:

- **04_Blender_MCP.md**